

Git, GitHub & Git Bash

Complete Command Cheat Sheet

1. Git — Setup & Configuration

<code>git config --global user.name "Name"</code>	Set your global username for commits
<code>git config --global user.email "you@x.com"</code>	Set your global email for commits
<code>git config --list</code>	Show all current Git configuration settings
<code>git config --global core.editor "code --wait"</code>	Set default editor for Git (e.g. VS Code)
<code>git config --global init.defaultBranch main</code>	Set default branch name for new repos
<code>git --version</code>	Check installed Git version
<code>git help <command></code>	Open help documentation for a command

2. Git — Starting a Repository

<code>git init</code>	Initialize a new local Git repository
<code>git init <dir></code>	Create a new directory and initialize it as a repo
<code>git clone <url></code>	Clone a remote repository to your local machine
<code>git clone <url> <folder></code>	Clone into a specific folder name
<code>git clone --depth 1 <url></code>	Shallow clone — only latest commit history

3. Git — Staging & Committing

<code>git status</code>	Show state of working directory and staging area
<code>git add <file></code>	Stage a specific file for commit
<code>git add .</code>	Stage all changed/new files in current directory
<code>git add -A</code>	Stage all changes including deletions, repo-wide
<code>git add -p</code>	Interactively stage parts (hunks) of changed files
<code>git commit -m "message"</code>	Commit staged changes with a message
<code>git commit -am "message"</code>	Stage tracked files and commit in one step
<code>git commit --amend</code>	Edit the most recent commit (message or content)
<code>git commit --amend --no-edit</code>	Add staged changes to last commit, keep message
<code>git rm <file></code>	Remove a file from working directory and stage removal
<code>git mv <old> <new></code>	Rename/move a file and stage the change

4. Git — Branching & Merging

<code>git branch</code>	List all local branches
<code>git branch -a</code>	List all local and remote branches
<code>git branch <name></code>	Create a new branch
<code>git branch -d <name></code>	Delete a branch (safe, checks merge status)
<code>git branch -D <name></code>	Force delete a branch
<code>git branch -m <new-name></code>	Rename current branch
<code>git checkout <branch></code>	Switch to an existing branch
<code>git checkout -b <branch></code>	Create and switch to a new branch
<code>git switch <branch></code>	Switch branches (modern alternative to checkout)
<code>git switch -c <branch></code>	Create and switch to a new branch (modern)
<code>git merge <branch></code>	Merge specified branch into current branch
<code>git merge --no-ff <branch></code>	Merge while forcing a merge commit
<code>git merge --abort</code>	Abort a merge in progress due to conflicts
<code>git rebase <branch></code>	Reapply commits on top of another base branch
<code>git rebase -i HEAD~n</code>	Interactive rebase of last n commits (squash/edit)
<code>git rebase --continue</code>	Continue rebase after resolving conflicts
<code>git rebase --abort</code>	Cancel an in-progress rebase
<code>git cherry-pick <commit></code>	Apply a specific commit onto current branch

5. Git — Remote Repositories

<code>git remote -v</code>	List remote repositories with URLs
<code>git remote add origin <url></code>	Add a new remote named 'origin'
<code>git remote remove <name></code>	Remove a remote connection
<code>git remote rename <old> <new></code>	Rename a remote
<code>git fetch</code>	Download objects/refs from remote (no merge)
<code>git fetch --all</code>	Fetch updates from all remotes
<code>git pull</code>	Fetch and merge changes from remote branch
<code>git pull --rebase</code>	Fetch and rebase local commits on top
<code>git push</code>	Push committed changes to remote branch
<code>git push -u origin <branch></code>	Push and set upstream tracking branch
<code>git push --force</code>	Force push (overwrites remote history — use carefully)
<code>git push --force-with-lease</code>	Safer force push, checks remote hasn't changed
<code>git push origin --delete <branch></code>	Delete a branch on the remote

6. Git — Inspecting History

<code>git log</code>	Show commit history
<code>git log --oneline</code>	Show condensed one-line-per-commit log
<code>git log --graph --oneline --all</code>	Visual graph of branch/commit history
<code>git log -p</code>	Show commit history with full diffs
<code>git log --author="name"</code>	Filter commits by author
<code>git log --since="2 weeks ago"</code>	Filter commits by date range
<code>git show <commit></code>	Show details and diff of a specific commit
<code>git diff</code>	Show unstaged changes vs last commit
<code>git diff --staged</code>	Show staged changes vs last commit
<code>git diff <branch1> <branch2></code>	Compare two branches
<code>git blame <file></code>	Show who last modified each line of a file

7. Git — Undoing Changes

<code>git restore <file></code>	Discard unstaged changes in working directory
<code>git restore --staged <file></code>	Unstage a file (keep changes in working dir)
<code>git checkout -- <file></code>	Legacy: discard changes in working directory
<code>git reset <file></code>	Unstage a file
<code>git reset --soft HEAD~1</code>	Undo last commit, keep changes staged
<code>git reset --mixed HEAD~1</code>	Undo last commit, keep changes unstaged (default)
<code>git reset --hard HEAD~1</code>	Undo last commit and discard all changes (destructive)
<code>git revert <commit></code>	Create a new commit that undoes a previous commit
<code>git clean -fd</code>	Remove untracked files and directories

8. Git — Stashing

<code>git stash</code>	Save uncommitted changes and clean working directory
<code>git stash -u</code>	Stash including untracked files
<code>git stash list</code>	List all stashed change sets
<code>git stash pop</code>	Apply and remove the most recent stash
<code>git stash apply</code>	Apply most recent stash but keep it in stash list
<code>git stash apply stash@{2}</code>	Apply a specific stash by index
<code>git stash drop</code>	Delete the most recent stash
<code>git stash clear</code>	Delete all stashes

9. Git — Tags

<code>git tag</code>	List all tags
<code>git tag <name></code>	Create a lightweight tag at current commit
<code>git tag -a <name> -m "msg"</code>	Create an annotated tag with message
<code>git push origin <tag></code>	Push a single tag to remote
<code>git push origin --tags</code>	Push all tags to remote
<code>git tag -d <name></code>	Delete a local tag
<code>git push origin --delete <tag></code>	Delete a tag on the remote

10. Git — Submodules

<code>git submodule add <url></code>	Add a repository as a submodule
<code>git submodule init</code>	Initialize submodules in cloned repo
<code>git submodule update</code>	Fetch submodule contents per recorded commit
<code>git submodule update --init --recursive</code>	Init and fetch all nested submodules

11. GitHub CLI (gh) — Auth & Setup

<code>gh auth login</code>	Authenticate the CLI with your GitHub account
<code>gh auth status</code>	Check current authentication status
<code>gh config set editor "code --wait"</code>	Set default editor for gh CLI

Repositories

<code>gh repo create <name></code>	Create a new GitHub repository
<code>gh repo clone <owner/repo></code>	Clone a repository via gh CLI
<code>gh repo fork <owner/repo></code>	Fork a repository to your account
<code>gh repo view --web</code>	Open current repo in the browser
<code>gh repo delete <owner/repo></code>	Delete a GitHub repository

Pull Requests

<code>gh pr create</code>	Create a new pull request interactively
<code>gh pr create --title "t" --body "b"</code>	Create a PR with title and body inline
<code>gh pr list</code>	List open pull requests
<code>gh pr view <number></code>	View details of a specific PR
<code>gh pr checkout <number></code>	Check out a PR branch locally
<code>gh pr merge <number></code>	Merge a pull request
<code>gh pr diff <number></code>	Show diff of a pull request
<code>gh pr review --approve</code>	Approve a pull request

Issues

<code>gh issue create</code>	Create a new issue interactively
<code>gh issue list</code>	List open issues
<code>gh issue view <number></code>	View details of a specific issue
<code>gh issue close <number></code>	Close an issue

12. GitHub — SSH Key Setup

<code>ssh-keygen -t ed25519 -C "you@x.com"</code>	Generate a new SSH key pair
<code>eval "\$(ssh-agent -s)"</code>	Start the SSH agent in current shell
<code>ssh-add ~/.ssh/id_ed25519</code>	Add your private key to the SSH agent
<code>cat ~/.ssh/id_ed25519.pub</code>	Print public key to copy into GitHub settings
<code>ssh -T git@github.com</code>	Test SSH connection to GitHub

13. GitHub — Common Workflow

<code>git clone git@github.com:user/repo.git</code>	Clone using SSH remote
<code>git checkout -b feature/my-feature</code>	Create feature branch for new work
<code>git push -u origin feature/my-feature</code>	Push feature branch and open it for a PR
<code>gh pr create --fill</code>	Open a PR using commit info to fill title/body
<code>git fetch origin && git merge origin/main</code>	Sync local branch with latest main

14. Git Bash — Navigation

<code>pwd</code>	Print current working directory
<code>ls</code>	List files and folders in current directory
<code>ls -la</code>	List all files (incl. hidden) with details
<code>cd <dir></code>	Change into a directory
<code>cd ..</code>	Move up one directory level
<code>cd ~</code>	Go to home directory
<code>cd -</code>	Go to previous directory

15. Git Bash — Files & Directories

<code>mkdir <dir></code>	Create a new directory
<code>mkdir -p a/b/c</code>	Create nested directories in one command
<code>touch <file></code>	Create an empty file or update its timestamp
<code>rm <file></code>	Delete a file
<code>rm -r <dir></code>	Delete a directory and its contents recursively
<code>rm -rf <dir></code>	Force delete directory recursively (no prompt)
<code>cp <src> <dest></code>	Copy a file
<code>cp -r <src> <dest></code>	Copy a directory recursively
<code>mv <src> <dest></code>	Move or rename a file/directory
<code>cat <file></code>	Print full contents of a file
<code>less <file></code>	View file contents page by page
<code>head -n 20 <file></code>	Show first 20 lines of a file
<code>tail -n 20 <file></code>	Show last 20 lines of a file
<code>tail -f <file></code>	Follow a file live as it's written to
<code>find . -name "*.js"</code>	Search recursively for files matching pattern

<code>grep -r "text" .</code>	Search recursively for a text string in files
-------------------------------	---

16. Git Bash — Permissions & System

<code>chmod +x <file></code>	Make a file executable
<code>chmod 755 <file></code>	Set specific read/write/execute permissions
<code>whoami</code>	Print current logged-in username
<code>history</code>	Show command history
<code>clear</code>	Clear the terminal screen
<code>alias gs='git status'</code>	Create a shortcut alias for a command
<code>which <command></code>	Show the path of an executable command
<code>export VAR=value</code>	Set an environment variable for the session
<code>echo \$VAR</code>	Print the value of a variable

17. Git Bash — Useful Keyboard Shortcuts

Tab	Auto-complete file/folder names and commands
Ctrl + C	Cancel/kill the current running command
Ctrl + L	Clear the terminal screen
Ctrl + A	Move cursor to beginning of the line
Ctrl + E	Move cursor to end of the line
Ctrl + R	Search through command history
Ctrl + U	Clear line before the cursor
Ctrl + K	Clear line after the cursor
Up / Down arrows	Cycle through previous commands

Tip: In Git Bash, most standard Unix/Linux commands work as shown above, since Git Bash emulates a Bash shell (via MinGW) on Windows.